

Learning Objectives

This lab lets you practice nested loops (including branching in the loops), and nested loops.

Estimated Size

1 program file: **zigzag.py**, 2 functions, and around 15-30 lines of code in each function.

Setup

In your Python files, you must include author information. On the first line, add a **# Author comment line**, where you mention your full name. Similarly, include **your email and Spire ID** on the **# Email** and **# Spire ID** comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author    : John Snow
# Email     : jsnow@umass.edu
# Spire ID  : 12345678
```

0. nested loop basics

A nested loop is a loop in the body of another loop. We refer to them as the **inner loop** and the **outer loop**. It sounds complicated but don't be scared!

The simplest nested loop is one where the inner loop iterations do not depend on the outer loop iterations. For example, the code below prints out a multiplication table. The outer loop runs over numbers from **1** to **9**. For each outer loop iteration, the inner loop also runs over **1** through **9**, independent of what the outer loop variable **i** is. So in total, the inner loop body will run **9x9=81** times. By default, the **print** function will end the print with a newline. The **end= ' '** argument tells it to end the print with a space character instead of a newline.

```
for i in range(1, 10):           # outer loop iterates over [1, 9]
    for j in range(1, 10):       # inner loop iterates over [1, 9]
        print(f'{i}x{j}={i*j}', end=' ') # inner loop body, print multiplication
    print()                     # print a new line
```

1. Implement the function **longest_zigzag_from_start** (8 points)

Given a list of integers, a **zigzag pattern** is a sequence where consecutive differences strictly alternate in sign (up → down → up ... or down → up → down ...).

Write a function, called **longest_zigzag_from_start**, which should:

- Take a list of integers `lst` as a parameter
- Return the length of the **longest zigzag subsequence that starts from the first element**
- The subsequence must start at index `0` and include consecutive elements
- A list of fewer than 2 elements counts as a zigzag of length equal to the list length

Important notes:

- You may use loops and conditionals.
- The function should return an **integer**.
- The return value is the length of the longest zigzag starting at the first element, not the actual subsequence.
- Do **NOT** ask the user for any input. Do **NOT** print anything in your function.

Below are examples of running a correct implementation of this function:

```
print(longest_zigzag_from_start([1, 3, 2, 4, 3])) # 5 → entire list is zigzag
print(longest_zigzag_from_start([1, 2, 3, 4, 5])) # 2 → only [1,2] is zigzag (up → not down)
print(longest_zigzag_from_start([3, 1, 4, 2, 5, 6])) # 5 → [3,1,4,2,5] is longest zigzag from start
print(longest_zigzag_from_start([10])) # 1 → single element
print(longest_zigzag_from_start([1, 3, 2, 1, 2, 3, 4])) # 3 → [1,3,2] zigzag from start
```

2. Implement function `zigzag_lengths_from_all_starts` (7 points)

A list of integers is called a **zigzag** if consecutive differences strictly alternate in sign (up → down → up ... or down → up → down ...).

Write a function, called `zigzag_lengths_from_all_starts`, which should:

- Take a list of integers `lst` as input
- Return a **list of integers**, where each element at index `i` is the length of the longest contiguous zigzag starting at position `i`
- Differences of `0` break the zigzag
- Do **NOT** ask the user for any input. Do **NOT** print anything in your function.

Below are examples of running a correct implementation of this function:

```
print(zigzag_lengths_from_all_starts([1, 3, 2, 4, 3])) # [5, 4, 3, 2, 1]
print(zigzag_lengths_from_all_starts([1, 2, 3, 4, 5])) # [2, 2, 2, 2, 1]
print(zigzag_lengths_from_all_starts([3, 1, 4, 2, 5, 6])) # [5, 4, 3, 2, 2, 1]
```

```
print(zigzag_lengths_from_all_starts([10])) # [1]
```

3. Submit your code to Gradescope

Log in to [Gradescope](#), select **Pre-Lab 06: (Code)**, and submit the required file. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline. **Only one person per group should submit.**

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions and will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the [UMass Academic Honesty Policy and Procedures](#) so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty you should re-read the course policies. We assume you have read the course policies in detail and by submitting this project you have provided your virtual signature in agreement with these policies.